R PROGRAMMING (35021P28)

UNIT III — Data Frames, Factors, Control Flow & Functions in R

Question Bank — Model Answers

Prepared by: Dr. S. Pitchumani Angayarkanni

Department of CSE, AVIT, VMRF Deemed University

Course Outcome: CO3 | All questions mapped per approved Unit III Question Bank

Section	Count	Marks Each	Bloom's Levels Covered
MCQs	20	1	Remember (L1), Understand (L2), Apply (L3)
Part B	20	6	Understand (L2), Apply (L3)
Part C	20	10	Understand (L2), Apply (L3)

Section 1 — Multiple Choice Questions (1 Mark Each)

Q.No	Question & Options	Correct Answer	Bloom's Level
1	**Which function creates a data frame in R?** **A. data.frame()** B. dataframe() C. frame.create() D. df()	**A. data.frame()**	Remember (L1)
2	**What does the merge() function do by default (all=FALSE)?** A. Left join **B. Inner join** C. Outer join D. Right join	**B. Inner join**	Understand (L2)
3	**Which function converts a character column to a factor in R?** A. factor() B. as.factor() **C. Both A and B** D. make.factor()	**C. Both A and B**	Remember (L1)
4	**What does table(x) return?** A. A data frame of x **B. A frequency table of x** C. A sorted vector D. A list of x	**B. A frequency table of x**	Understand (L2)
5	**In R, what does the <<- operator do?** A. Assigns within local scope **B. Assigns to a variable in the parent/global scope** C. Compares two values D. Deletes a variable	**B. Assigns to a variable in the parent/global scope**	Understand (L2)
6	**Which control statement is used to skip the current iteration in an R loop?** A. break **B. next** C. continue D. skip	**B. next**	Remember (L1)
7	**What is the output of: is.function(mean) in R?** A. FALSE **B. TRUE**	**B. TRUE**	Understand (L2)

Q.No	Question & Options	Correct Answer	Bloom's Level
	C. NaN D. Error		
8	**Which function applies a function across a factor's groups in a vector?** A. sapply() B. lapply() **C. tapply()** D. apply()	**C. tapply()**	Apply (L3)
9	**What is the result of: set.seed(42); sample(1:5,3)?** A. Always different **B. Always the same reproducible result** C. Returns NA D. Throws an error	**B. Always the same reproducible result**	Understand (L2)
10	**Which function is used for cross-tabulation of two factors in R?** A. table() B. xtabs() **C. Both A and B** D. crosstab()	**C. Both A and B**	Apply (L3)
11	**Which function returns the number of rows in a data frame?** **A. nrow()** B. length() C. rowcount() D. dim.rows()	**A. nrow()**	Remember (L1)
12	**What does levels(factor(c('Low','High','Low'))) return?** **A. c('Low','High')** B. c('High','Low','Low') C. c('Low','High','Low') D. NaN	**A. c('Low','High')**	Understand (L2)
13	**Which loop structure guarantees at least one execution in R?** A. for B. while **C. repeat** D. next	**C. repeat**	Remember (L1)
14	**What is a default argument**	**B. An argument with a pre-**	Understand (L2)

Q.No	Question & Options	Correct Answer	Bloom's Level
	in an R function definition? A. An argument that must always be supplied **B. An argument with a pre-set value used if not supplied** C. An argument that is ignored D. A required first argument	**set value used if not supplied**	
15	**What does the function do.call() do in R?** **A. Calls a function using a list of arguments** B. Deletes a function C. Creates a function dynamically D. Calls a function recursively	**A. Calls a function using a list of arguments**	Apply (L3)
16	**Which Boolean operator performs short-circuit evaluation on single values?** A. & B. | **C. &&** D. %in%	**C. &&**	Understand (L2)
17	**Which function is used to apply a function over rows of a data frame?** **A. apply(df, 1, FUN)** B. apply(df, 2, FUN) C. rowFUN(df) D. sapply(df, FUN, row=TRUE)	**A. apply(df, 1, FUN)**	Apply (L3)
18	**What does an ordered factor in R allow that a regular factor does not?** A. Conversion to numeric **B. Comparison operators like < and >** C. Removing NA values D. Sorting alphabetically only	**B. Comparison operators like < and >**	Understand (L2)
19	**Which function generates random numbers from a uniform distribution in R?** A. rnorm() **B. runif()** C. rpois() D. rbinom()	**B. runif()**	Remember (L1)
20	**What is the purpose of the**	**B. Select elements of a**	Apply (L3)

Q.No	Question & Options	Correct Answer	Bloom's Level
	Filter() function in R? A. Sort a list **B. Select elements of a list/vector matching a predicate function** C. Remove duplicates D. Merge two lists	**list/vector matching a predicate function**	

Section 2 — Part B Descriptive Answers (6 Marks Each)

Q3.11 *(6 Marks | Bloom's: Understand (L2) | CO3)*

Explain the creation of data frames in R. Illustrate matrix-like operations on data frames with examples.

A data frame is R's core rectangular data structure for tabular, mixed-type data: it is essentially a list of equal-length vectors, where each column can be a different type (numeric, character, logical, factor) but every value within one column stays the same type. It is created with data.frame(), which builds the frame column by column, and its shape can be inspected with str(), nrow(), ncol() and dim().

```
students <- data.frame(
  id     = 1:4,
  name   = c("Asha","Ravi","Meera","Kabir"),
  marks  = c(78, 85, 92, 67),
  passed = c(TRUE, TRUE, TRUE, FALSE)
)
str(students)
nrow(students); ncol(students)
```

By default, stringsAsFactors is FALSE in modern R, so the name column stays character rather than being auto-converted to a factor.

Because a data frame is internally rectangular, it also supports matrix-style [row, col] indexing, filtering with logical conditions, and row/column binding:

```
students[2, ]                # 2nd row, all columns
students[, "marks"]          # marks column only
students[students$marks > 75, ]   # filter rows

new_row <- data.frame(id=5, name="Zoya", marks=88, passed=TRUE)
students <- rbind(students, new_row)   # add a row
colMeans(students["marks"])            # column-wise mean
```

rbind() appends rows and cbind() appends columns, both preserving each column's original type. Arithmetic and summary functions such as colSums(), rowSums() and colMeans() operate directly on numeric-only data frames, exactly as they would on a matrix, which is why data frames are described as supporting matrix-like operations while still allowing heterogeneous column types.

Q3.12 *(6 Marks | Bloom's: Apply (L3) | CO3)*

Describe the merge() function in R with examples for inner, outer, left, and right joins.

merge() combines two data frames on a common key column, exactly like a SQL join. The join type is controlled by the all, all.x and all.y arguments.

```
students <- data.frame(id=1:4, name=c("Asha","Ravi","Meera","Kabir"))
dept <- data.frame(id=c(1,2,3), dept=c("CSE","AIML","ECE"))

merge(students, dept, by="id")               # inner join
merge(students, dept, by="id", all=TRUE)      # outer join
merge(students, dept, by="id", all.x=TRUE)    # left join
merge(students, dept, by="id", all.y=TRUE)    # right join
```

Inner join (all=FALSE, the default): only rows whose id exists in both frames are kept (ids 1-3 here; Kabir with id 4 is dropped since he has no matching department).

Outer join (all=TRUE): every id from both frames is kept; unmatched columns are filled with NA (Kabir's dept becomes NA).

Left join (all.x=TRUE): every row of students (the left/first frame) is kept, with NA where dept has no match.

Right join (all.y=TRUE): every row of dept (the right/second frame) is kept, with NA where students has no match. Choosing the right join type depends on whether unmatched rows from one, both, or neither table should be preserved in the result.

Q3.13 *(6 Marks | Bloom's: Understand (L2) | CO3)*

Explain factors in R: creation, levels, ordered factors, and conversion using as.factor() with examples.

A factor stores categorical data as underlying integer codes together with a levels attribute holding the category labels. Factors are created either with factor() on a plain vector or with as.factor() to coerce an existing character/numeric vector.

```
grade <- factor(c("Low","High","Low","Med"))
levels(grade)          # "High" "Low" "Med" (alphabetical by default)
as.integer(grade)      # underlying integer codes

risk <- factor(c("Low","High","Med"),
               levels=c("Low","Med","High"), ordered=TRUE)
risk[1] < risk[3]      # TRUE: Low < High
```

levels() returns the distinct category names in the order R has assigned internally (alphabetical unless levels= is specified). An ordered factor, created with ordered=TRUE and an explicit levels= sequence, additionally supports comparison operators such as < and >, which a regular (unordered) factor does not, because R now knows the rank order of the categories.

as.factor(x) is the general-purpose way to convert any vector — most commonly a character column read in from a CSV — into a factor without manually specifying category order; it is functionally equivalent to factor(x) when no custom levels or ordering is required.

Q3.14 *(6 Marks | Bloom's: Apply (L3) | CO3)*

Explain the table() function in R. How is it used for frequency tables and cross-tabulation?

table() builds a frequency count of each distinct value/level in one or more categorical vectors. Passed a single vector, it produces a one-way frequency table; passed two (or more) vectors, it produces a cross-tabulation (contingency table) showing the joint distribution.

```
gender <- c("F","M","F","F","M","M")
result <- c("Pass","Pass","Fail","Pass","Fail","Pass")

table(gender)                # one-way frequency table
t <- table(gender, result)   # two-way cross-tabulation
prop.table(t, margin=1)      # row-wise proportions
addmargins(t)                # append row/column totals
```

table(gender) alone reports how many F's and M's there are. table(gender, result) cross-tabulates the two factors into a small matrix showing, for each gender, how many passed and how many failed — the joint frequency distribution. prop.table() converts the raw counts into proportions (row-wise with margin=1, column-

wise with margin=2, or overall with no margin), and addmargins() appends row and column totals for a quick summary report. Together these make table() the standard tool for summarising categorical survey or experimental data in R.

Q3.15 *(6 Marks | Bloom's: Apply (L3) | CO3)*

Explain control statements in R: if-else, for, while, repeat, break, and next with examples.

Control statements determine the order in which R executes instructions, letting a program make decisions, repeat tasks and alter its normal top-to-bottom flow.

```
# if-else: choose a branch
x <- 5
if (x > 10) print("Greater") else print("Not greater")

# for: known number of iterations
for (i in 1:5) print(i^2)

# while: condition tested before each pass
n <- 5
while (n > 0) { print(n); n <- n - 1 }

# repeat + break: at least one execution, exits explicitly
repeat { print("runs once"); break }

# next: skip an iteration
for (i in 1:10) { if (i %% 2 == 0) next; print(i) }
```

if / if-else picks between alternative code paths based on a logical condition (pass/fail, eligible/not eligible type decisions). for iterates a fixed number of times over a known sequence, vector or list. while repeats as long as its condition stays TRUE, checked before every iteration, and may execute zero times. repeat loops indefinitely with no header condition and is guaranteed to run at least once — it must be stopped explicitly with break. break terminates the innermost loop immediately, while next skips only the remainder of the current iteration and moves on to the next one; both work inside for, while and repeat.

Q3.16 *(6 Marks | Bloom's: Understand (L2) | CO3)*

Describe arithmetic and Boolean operators in R. What are short-circuit operators && and ||?

R's arithmetic operators (+, -, *, /, ^, %% for modulus, %/% for integer division) and comparison operators (==, != , <, >, <=, >=) are all vectorised — they operate element-by-element on entire vectors and return a result of the same length, which is what makes them so convenient for filtering rows of a data frame (e.g. students[students$marks > 75,]).

```
c(1,2,3) + c(10,20,30)     # element-wise arithmetic
c(1,2,3) > 2               # element-wise comparison -> logical vector

x <- numeric(0)
if (length(x) > 0 && x[1] > 5) print("safe")   # && never evaluates x[1]
```

The element-wise Boolean operators & and | compare vectors position by position and return a logical vector, so they are the correct choice for subsetting data frame rows. The short-circuit operators && and || work on single (length-1) logical values only, evaluate left to right, and stop as soon as the overall result is certain — the second operand may never even run. This makes && the safe and required choice inside if() conditions, for

example to check length(x) > 0 before safely indexing x[1] without risking an error. %in% tests membership and xor(a,b) returns TRUE only when exactly one of a, b is TRUE.

Q3.17 *(6 Marks | Bloom's: Apply (L3) | CO3)*

Explain default values for arguments in R functions. How does R handle missing arguments?

Any parameter in an R function definition can be given a default value using the syntax name = value. If the caller omits that argument, R automatically substitutes the default; if the caller supplies a value, it overrides the default.

```
greet <- function(name, greeting="Hello") paste(greeting, name)
greet("Meera")              # "Hello Meera"
greet("Meera", "Welcome")   # "Welcome Meera"

power <- function(x, n=2) x^n
power(5)     # 25 (n defaults to 2, i.e. square)
```

Default values can even reference other parameters of the same function, and are evaluated lazily — only when the parameter is actually used inside the function body, not at the moment the call is made. If a required argument (one with no default) is genuinely missing when the function body tries to use it, R raises an error such as "argument \"x\" is missing, with no default"; the missing() function can be used inside a function body to test explicitly whether a given argument was supplied by the caller. This mechanism supports flexible, backward-compatible function signatures, since new optional parameters can be added later without breaking existing calls.

Q3.18 *(6 Marks | Bloom's: Understand (L2) | CO3)*

Explain environment and scope in R. What is lexical scoping? How does R resolve variable names?

Every function call in R creates its own local environment holding that call's local variables, which disappear once the call ends. R uses lexical (static) scoping: when a function needs a free variable that is not defined locally, it looks in the environment where the function was DEFINED, not the environment from which it happens to be called.

```
x <- 10
f <- function() {
  x <- 20
  g <- function() print(x)   # finds x in f's enclosing environment
  g()
}
f()          # prints 20
print(x)     # global x is unchanged -> 10
```

Here g() is defined inside f(), so its enclosing environment is f()'s local environment, where x is 20 — that is the value g() finds and prints, regardless of where g() might later be called from. If a name is not found in the local environment, R searches successively enclosing environments, up through the global environment and finally the search path (loaded packages). environment(f) can be used to inspect which environment a given function is bound to. This lexical model is what makes closures possible: an inner function permanently "remembers" the environment in which it was created, even after the outer function has finished executing.

Q3.19 *(6 Marks | Bloom's: Apply (L3) | CO3)*

Explain recursion in R with factorial and Fibonacci examples. What are the risks of deep recursion?

A recursive function calls itself with a progressively smaller version of the problem until it reaches a base case that stops further calls. Every correct recursive function needs exactly two parts: a base case (stopping condition) and a recursive case that moves the problem closer to that base case.

```
factorial <- function(n) {
  if (n <= 1) return(1)        # base case
  n * factorial(n - 1)         # recursive case
}
factorial(5)     # 120

fib <- function(n) if (n <= 1) n else fib(n-1) + fib(n-2)
sapply(0:6, fib) # 0 1 1 2 3 5 8
```

In factorial(5), each call multiplies n by the result of factorial(n-1) until n<=1 returns 1, unwinding to 5*4*3*2*1 = 120. In fib(), each call branches into two smaller recursive calls, which is elegant but re-computes the same sub-results repeatedly, making naive recursive Fibonacci exponential in time complexity.

The main risk of deep recursion is exhausting R's default call-stack limit (governed by options("expressions")); very deep or unbounded recursion causes a "C stack usage too close to the limit" error and crashes the call. Recursive functions are also generally slower and more memory-hungry than an equivalent iterative (loop-based) version because every pending call frame must stay on the stack. For this reason, deeply recursive algorithms are often rewritten iteratively, or use memoization/accumulator techniques to avoid repeated re-computation, as with Fibonacci.

Q3.20 *(6 Marks | Bloom's: Apply (L3) | CO3)*

Describe Math and Simulations in R: rnorm(), runif(), set.seed(), sample(), and probability functions.

R has strong built-in support for generating random numbers and running simulations. runif(n, min, max) draws n values from a continuous uniform distribution; rnorm(n, mean, sd) draws from a normal (Gaussian) distribution; related functions rpois() and rbinom() generate counts from the Poisson and Binomial distributions respectively.

```
set.seed(42)
sample(1:5, 3)              # random draw without replacement
runif(3, min=0, max=10)     # 3 uniform random numbers
rnorm(3, mean=50, sd=5)     # 3 normal random numbers
```

sample(x, size) draws a random subset of size elements from vector x, with or without replacement (replace=TRUE/FALSE). set.seed(x) fixes R's pseudo-random number generator to a specific starting state, so that every subsequent random draw becomes exactly reproducible across runs — essential for debugging simulations and for grading/verifying student output. Together, these functions underpin R's simulation capabilities, most notably Monte Carlo methods, where repeated random sampling is used to estimate probabilities or numeric quantities that are hard to compute analytically, such as estimating pi or modelling a coin-toss or random-walk experiment.

Q3.21 *(6 Marks | Bloom's: Understand (L2) | CO3)*

Explain how functions are objects in R. What does this mean for passing functions as arguments?

In R, a function is a first-class value, exactly like a number or a string: it can be assigned to a variable, stored inside a list, passed as an argument to another function, and returned as the result of another function.

```
is_even <- function(n) n %% 2 == 0
```

```
is.function(is_even)        # TRUE

sapply(1:6, is_even)        # a function passed directly as an argument
Filter(is_even, 1:10)       # is_even used as a predicate
```

Because functions are objects, R supports passing functions directly into higher-order functions such as apply(), sapply(), lapply(), Map(), Reduce() and Filter() — instead of writing an explicit loop, the desired operation is simply handed in as a value. This enables function factories: functions that build and return other, customised functions (closures), and anonymous (unnamed) functions written inline, e.g. sapply(1:5, function(x) x^2). Treating functions as ordinary objects is the foundation of R's functional-programming style, letting concise, composable code replace repetitive loop-based code.

Q3.22 *(6 Marks | Bloom's: Apply (L3) | CO3)*

Describe 'writing upstairs' using <<- in R. Explain with a closure-based counter function example.

'Writing upstairs' refers to the <<- superassignment operator, which assigns a value to a variable in an ENCLOSING (parent or global) environment instead of the current local one. R searches outward through enclosing environments for an existing binding of that name; if none is found anywhere, <<- creates the variable in the global environment as a fallback.

```
make_counter <- function() {
  count <- 0
  function() {
    count <<- count + 1   # writes to count in the enclosing environment
    count
  }
}
counter <- make_counter()
counter()   # 1
counter()   # 2
counter()   # 3
```

Here make_counter() creates a local variable count and returns an inner (anonymous) function. That inner function forms a closure — it retains a permanent reference to make_counter()'s environment even after make_counter() has finished running. Each time counter() is called, <<- modifies count in that retained enclosing environment rather than creating a fresh local count each time, so the value persists and increments across separate calls: 1, then 2, then 3. This is the standard R mechanism for stateful objects such as counters, accumulators and memoized caches. Because it mutates state outside the current function, <<- should be used sparingly, as it breaks strict function purity and can make code harder to reason about.

Q3.23 *(6 Marks | Bloom's: Understand (L2) | CO3)*

Explain replacement functions in R (e.g., names<-, dim<-). How are they defined and used?

A replacement function is a function whose name ends in <- and which modifies part of an object while appearing to the user as an ordinary assignment, e.g. names(x) <- v or dim(x) <- c(2,3). Internally, R rewrites the expression names(x) <- v as x <- `names<-`(x, v) — the assignment-style syntax is essentially syntactic sugar over a normal function call.

```
`second<-` <- function(x, value) {
  x[2] <- value
  x
}
```

```
v <- c(10, 20, 30)
second(v) <- 99
v       # 10 99 30
```

Two rules govern a valid replacement function: its LAST parameter must be named value (this is where R plugs in the right-hand side of the assignment), and the function must return the ENTIRE modified object, not just the changed part, because the returned value replaces x wholly. Built-in replacement functions such as names<-, levels<-, dim<- and colnames<- all follow this same pattern. Writing custom replacement functions lets a user-defined object support the same natural assignment-style syntax as base-R structures, which is considered good, idiomatic R design.

Q3.24 *(6 Marks | Bloom's: Apply (L3) | CO3)*

Apply functions to data frames using apply(), lapply(), and sapply(). Give examples.

R provides a family of apply-type functions for running a function repeatedly across a data frame without writing an explicit loop, differing mainly in what shape of result they return.

```
students <- data.frame(id=1:4, marks=c(78,85,92,67), extra=c(5,3,9,1))

apply(students[c("marks","extra")], 1, sum)   # row-wise totals
apply(students[c("marks","extra")], 2, mean)  # column-wise means

lapply(students["marks"], mean)     # always returns a list
sapply(students["marks"], mean)     # simplifies to a vector
```

apply(df, MARGIN, FUN) works on a data frame that is coercible to a matrix (usually all-numeric columns): MARGIN=1 applies FUN across each row, MARGIN=2 applies it down each column. lapply(df, FUN) applies FUN to every column of df and always returns a list, which is useful when the results differ in type or length across columns — it never tries to simplify the output. sapply(df, FUN) does the same column-wise application as lapply() but automatically simplifies the result into a vector or matrix whenever that is possible, making it the more convenient choice for quick numeric summaries such as sapply(students["marks"], mean).

Q3.25 *(6 Marks | Bloom's: Apply (L3) | CO3)*

Describe tools for composing function code in R: do.call(), Reduce(), Map(), and Filter() with examples.

These four functions let R code replace explicit loops with a concise, functional-programming style.

```
# do.call: call a function using a list of arguments
do.call(sum, list(1,2,3))            # 6

# Reduce: successively combine elements into one value
Reduce(function(a,b) a+b, 1:5)       # 15

# Map: apply a function element-wise across parallel vectors
Map(function(x,y) x+y, 1:3, 4:6)     # list(5,7,9)

# Filter: keep elements matching a predicate
Filter(function(x) x %% 2 == 0, 1:10) # 2 4 6 8 10
```

do.call(fun, args_list) invokes fun using a list of arguments assembled at run time — useful when the number or names of arguments are only known dynamically, e.g. do.call(rbind, list_of_data_frames). Reduce(fun, x) successively applies a binary function across the elements of x, collapsing a vector down to a single accumulated

value (like a running total). Map(fun, x, y, ...) applies fun element-by-element across one or more vectors/lists in parallel, always returning a list. Filter(predicate, x) keeps only the elements of x for which predicate(x) is TRUE, replacing a for-loop with an if inside it. Together they form R's core toolkit for building compact, loop-free data-processing pipelines.

Q3.26 *(6 Marks | Bloom's: Understand (L2) | CO3)*

Explain ordered factors in R. How are they used in comparison and statistical analysis?

An ordered factor is a factor created with ordered=TRUE (or via factor(..., ordered=TRUE)) whose levels have a meaningful, explicitly-defined rank order — for example Low < Med < High rather than a simple unordered set of categories.

```
risk <- factor(c("Low","High","Med"),
               levels=c("Low","Med","High"), ordered=TRUE)
risk[1] < risk[3]     # TRUE: Low < High
summary(risk)         # counts per level, in rank order
is.ordered(risk)      # TRUE
```

Unlike a regular factor, an ordered factor supports the comparison operators <, >, <=, >= directly, because R can compare the internal rank codes of its levels rather than just testing equality. This makes ordered factors appropriate for Likert-scale survey responses, risk categories, grade bands, and similar naturally-ranked categorical data. In statistical analysis, ordered factors are used in ordinal regression models, in sorting/summary functions that respect the defined rank rather than alphabetical order, and in visualisations (such as boxplots or bar charts) where the category axis should follow the natural Low→High progression instead of arbitrary alphabetical ordering.

Q3.27 *(6 Marks | Bloom's: Apply (L3) | CO3)*

Write an R program using a while loop to compute factorial iteratively. Compare with recursive approach.

```
# Iterative (while loop) factorial
factorial_iter <- function(n) {
  result <- 1
  while (n > 1) {
    result <- result * n
    n <- n - 1
  }
  result
}
factorial_iter(5)   # 120

# Recursive factorial
factorial_rec <- function(n) {
  if (n <= 1) return(1)
  n * factorial_rec(n - 1)
}
factorial_rec(5)    # 120
```

Both versions produce the identical result (120 for n=5), but they work very differently. The iterative version uses a single local variable result and a while loop that keeps multiplying and decrementing n until the condition n > 1 becomes FALSE; it uses constant (O(1)) extra stack space regardless of n. The recursive version instead expresses the same computation as n multiplied by the factorial of a smaller problem, with each nested call remaining on the call stack until the base case n<=1 is reached, then unwinding — this uses O(n) stack space and risks hitting R's call-stack limit for very large n.

In general, the iterative while-loop approach is more memory-efficient and faster for large n, while the recursive approach is often considered more elegant and closer to the mathematical definition, and is preferred when the problem naturally decomposes into smaller identical sub-problems (as with tree traversals or Tower of Hanoi).

Q3.28 *(6 Marks | Bloom's: Apply (L3) | CO3)*

Describe the use of Boolean operators (!, &, |, xor) in data frame filtering with examples.

Boolean operators are used to build and combine logical conditions for filtering rows of a data frame — each operator returns a logical vector the same length as the data frame, which is then used inside [ ] to select the matching rows.

```
students <- data.frame(id=1:5, marks=c(78,42,91,55,35),
active=c(TRUE,TRUE,FALSE,TRUE,FALSE))

students[students$marks > 60 & students$active, ]     # AND: both conditions
students[students$marks > 60 | students$marks < 40, ]  # OR: either condition
students[!students$active, ]                           # NOT: negate a condition
students[xor(students$marks > 60, students$active), ]  # exactly one is TRUE
```

! negates a logical vector, so !students$active selects inactive students. & performs an element-wise AND, keeping rows where both conditions hold (marks above 60 AND currently active); | performs an element-wise OR, keeping rows where at least one condition holds. xor(a, b) returns TRUE only where exactly one of the two conditions is true and the other is false — useful for detecting a mismatch between two flags (e.g. high marks but not active, or active but low marks). Because & and | are vectorised (unlike && and ||), they are the correct choice whenever the condition needs to be evaluated row-by-row across an entire data frame column.

Q3.29 *(6 Marks | Bloom's: Apply (L3) | CO3)*

Explain cross-tabulation using table() and xtabs(). Create a two-way frequency table from a sample dataset.

Cross-tabulation summarises the joint distribution of two (or more) categorical variables by counting how often each combination of their levels occurs together. R offers two equivalent ways to build this: table(), which takes the categorical vectors directly, and xtabs(), which uses a formula interface and works directly with data frame columns.

```
gender <- c("F","M","F","F","M","M")
result <- c("Pass","Pass","Fail","Pass","Fail","Pass")
survey <- data.frame(gender, result)

table(gender, result)              # vector-based cross-tabulation
xtabs(~ gender + result, data=survey)  # formula-based, same result
```

Both calls produce the same 2x2 contingency table: result gender Fail Pass F 1 2 M 1 2

table() is convenient for a small number of loose vectors, while xtabs() is preferred when the variables already live as columns inside a data frame and especially for building multi-way (3+ factor) tables, since the formula syntax (~ f1 + f2 + f3) scales more cleanly than nesting multiple arguments in table(). Once built, the table can be further summarised with prop.table() for proportions or addmargins() for row/column totals.

Q3.30 *(6 Marks | Bloom's: Apply (L3) | CO3)*

Describe Monte Carlo simulation in R to estimate pi using uniform random numbers. Write the program.

A Monte Carlo simulation uses repeated random sampling to approximate a numeric answer that would otherwise require analytical computation. Estimating pi is a classic example: points are scattered uniformly at random inside a unit square, and the fraction that lands inside the inscribed quarter-circle approximates the ratio of the circle's area to the square's area, which is pi/4.

```
set.seed(1)
n <- 100000
x <- runif(n)                       # random x-coordinates in [0,1]
y <- runif(n)                       # random y-coordinates in [0,1]
inside <- sum(x^2 + y^2 <= 1)        # points inside the quarter circle
pi_est <- 4 * inside / n
pi_est   # approximately 3.14 ...
```

runif(n) generates n uniformly distributed random x and y coordinates, each between 0 and 1. A point (x, y) lies inside the quarter unit circle if x^2 + y^2 <= 1; inside counts how many of the n points satisfy this. Since the quarter-circle's area is pi/4 relative to the unit square's area of 1, multiplying the observed fraction inside/n by 4 gives an estimate of pi. set.seed(1) makes the simulation reproducible, and as n grows larger, the estimate converges closer to the true value of pi (approximately 3.14159), which illustrates the general Monte Carlo pattern of simulate → count → estimate that also underlies coin-toss and random-walk simulations.

Section 3 — Part C Descriptive Answers (10 Marks Each)

Q3.31 *(10 Marks | Bloom's: Understand (L2) | CO3)*

Explain data frames in R comprehensively: creation, access, modification, matrix-like operations, merging, and applying functions. Provide examples for each operation.

A data frame is R's rectangular data structure for tabular, mixed-type data: a list of equal-length column vectors, each of its own type, sharing the same number of rows — conceptually a spreadsheet or database table. It is the standard input format for statistical functions, ML pipelines, and datasets imported via read.csv().

1. Creation:

```
students <- data.frame(
  id = 1:4, name = c("Asha","Ravi","Meera","Kabir"),
  marks = c(78,85,92,67), passed = c(TRUE,TRUE,TRUE,FALSE)
)
str(students); nrow(students); ncol(students)
```

stringsAsFactors defaults to FALSE in modern R, so character columns stay character. str() and summary() give a quick structural/statistical snapshot; nrow(), ncol() and dim() report its shape.

2. Access:

```
students$marks         # column access by name
students[, "marks"]    # equivalent, matrix-style
students[2, ]           # 2nd row, all columns
students[students$marks > 75, ]   # filter rows by condition
```

3. Modification:

```
students$grade <- ifelse(students$marks >= 80, "A", "B")   # add a column
students[1, "marks"] <- 80                                  # edit a cell
new_row <- data.frame(id=5, name="Zoya", marks=88, passed=TRUE, grade="A")
students <- rbind(students, new_row)                        # add a row
```

4. Matrix-like operations: rbind() adds rows, cbind() adds columns, both preserving column types. Arithmetic works column-wise on numeric columns; colSums(), rowSums(), colMeans() work directly on numeric-only frames, exactly as on a matrix.

5. Merging:

```
dept <- data.frame(id=c(1,2,3), dept=c("CSE","AIML","ECE"))
merge(students, dept, by="id")             # inner join (default)
merge(students, dept, by="id", all=TRUE)   # outer join
```

6. Applying functions:

```
sapply(students["marks"], mean)            # column-wise mean, simplified
apply(students[c("marks")], 1, function(x) x)  # row-wise application
lapply(students["marks"], summary)         # always returns a list
```

Together these six operations — create, access, modify, matrix-style manipulate, merge, and apply — cover the full lifecycle of working with tabular data in R, and are the foundation for every larger data-processing task in this unit.

Q3.32 *(10 Marks | Bloom's: Apply (L3) | CO3)*

Describe all join types using merge() in R (inner, outer, left, right). Write programs for each and explain when to use which join.

merge() combines two data frames on a common key column, mirroring SQL join semantics. The join type is chosen with the all, all.x and all.y parameters.

```
students <- data.frame(id=1:4, name=c("Asha","Ravi","Meera","Kabir"))
dept     <- data.frame(id=c(1,2,3), dept=c("CSE","AIML","ECE"))

# Inner join: only matching ids kept
merge(students, dept, by="id")

# Outer join: all ids from both, NA where unmatched
merge(students, dept, by="id", all=TRUE)

# Left join: all rows of students kept
merge(students, dept, by="id", all.x=TRUE)

# Right join: all rows of dept kept
merge(students, dept, by="id", all.y=TRUE)
```

Inner join (all=FALSE, default) keeps only rows whose key exists in BOTH frames — here ids 1-3, since Kabir (id 4) has no department. Use it when only fully-matched records are meaningful, e.g. computing per-department averages that require a valid department.

Outer join (all=TRUE) keeps every id from both frames, filling NA where a match is missing — Kabir appears with dept=NA. Use it for auditing or reconciliation, when it matters to see unmatched records from either side.

Left join (all.x=TRUE) keeps every row of the first (left) frame regardless of a match, filling NA in columns from the second frame — every student appears, with dept=NA if unassigned. Use it when the left table is the primary entity of interest and you simply want to enrich it optionally with data from the right table.

Right join (all.y=TRUE) is the mirror image: every row of the second (right) frame is kept, with NA filled for unmatched columns from the left. Use it when the right table is the entity of primary interest, e.g. listing every department even those with currently no students.

Choosing the correct join type is important because it directly determines whether unmatched records are silently dropped (inner), or preserved with NA placeholders (outer/left/right) — a common source of subtle bugs when the wrong join is used in a data pipeline.

Q3.33 *(10 Marks | Bloom's: Apply (L3) | CO3)*

Explain factors and tables in R in detail: creation, levels, ordered factors, table(), prop.table(), and cross-tabulation. Provide examples.

A factor represents categorical data as integer codes plus a levels attribute of category labels — R's structure for grouping variables such as gender, department or grade band.

1. Creation and levels:

```
grade <- factor(c("Low","High","Low","Med"))
```

```
levels(grade)        # distinct categories, alphabetical by default
nlevels(grade)       # count of distinct levels
as.integer(grade)    # underlying integer codes
as.factor(c("A","B","A"))  # coercion form
```

2. Ordered factors:

```
risk <- factor(c("Low","High","Med"),
     levels=c("Low","Med","High"), ordered=TRUE)
risk[1] < risk[3]     # TRUE, Low < High — comparison enabled by ordering
```

Ordered factors additionally support <, > comparisons because R knows the rank of each level; regular factors do not.

3. table() and frequency counting:

```
table(grade)          # one-way frequency count of each level
```

4. prop.table():

```
gender <- c("F","M","F","F","M","M")
result <- c("Pass","Pass","Fail","Pass","Fail","Pass")
t <- table(gender, result)
prop.table(t, margin=1)   # row-wise proportions (per gender)
prop.table(t)             # overall proportions
```

5. Cross-tabulation:

```
xtabs(~ gender + result, data=data.frame(gender, result))  # formula form
addmargins(t)     # append row and column totals
```

table(gender, result) and the formula-based xtabs() both produce the same two-way contingency table, revealing the joint frequency of gender and pass/fail outcome. prop.table() converts these raw counts to proportions for easier comparison across groups of different sizes, while addmargins() adds row/column totals for a quick summary report. Together, factors, table() and prop.table() form R's standard toolkit for encoding and summarising categorical survey and experimental data.

Q3.34 *(10 Marks | Bloom's: Apply (L3) | CO3)*

Write a comprehensive R program demonstrating all control statements: if-else, for, while, repeat, break, next with a real-world scenario for each.

Scenario: processing a class's exam marks to classify results, print a countdown, retry an unstable operation, and skip absentee records.

```
marks <- c(78, 42, 91, 35, 66, 84, NA, 55)

# 1. if-else: classify pass/fail
for (m in marks) {
  if (is.na(m)) next                      # 5. next: skip absentees
  if (m >= 50) {
    print(paste(m, "-> Pass"))
```

```
  } else {
    print(paste(m, "-> Fail"))
  }
}

# 2. for: known number of iterations, process every mark
total <- 0
for (m in marks) {
  if (is.na(m)) next
  total <- total + m
}
print(paste("Total of valid marks:", total))

# 3. while: countdown before results are announced
countdown <- 3
while (countdown > 0) {
  print(paste("Announcing results in", countdown))
  countdown <- countdown - 1
}

# 4. repeat + break: retry a flaky data read until it succeeds (max 5 tries)
attempt <- 1
repeat {
  print(paste("Attempt", attempt, "to read result sheet"))
  success <- (attempt == 3)     # simulate success on 3rd try
  if (success || attempt >= 5) break
  attempt <- attempt + 1
}
```

if-else classifies each valid mark as Pass or Fail — the core decision-making tool, ideal when there are exactly two mutually exclusive outcomes. for iterates over the fixed-length marks vector exactly once per element, appropriate because the number of records is known in advance. while implements a countdown, testing its condition before each pass and naturally handling the case of zero iterations if countdown already started at 0. repeat with break models a retry loop for an operation (such as reading an external data source) whose success time is not known in advance — it must always run at least once, with an explicit break once the exit condition (success or max attempts) is met. next is used throughout to skip NA (absentee) records without terminating the surrounding loop, letting the remaining data continue processing cleanly. This combination — decision, fixed iteration, condition-controlled iteration, indefinite iteration, and flow-skipping — covers every category of control statement examinable in the unit.

Q3.35 *(10 Marks | Bloom's: Understand (L2) | CO3)*

Explain R's scoping rules (lexical scoping) in detail. Illustrate environment, scope, <<-, and how R resolves free variables across nested functions.

R uses lexical (also called static) scoping: a function resolves any free variable (one it did not define locally) by looking in the environment where the function was DEFINED, not the environment from which it happens to be called. Each function call creates a brand-new local environment for its own variables, which is discarded once the call returns.

```
x <- 10                       # global environment
outer_fn <- function() {
  x <- 20                      # local to outer_fn's call
  inner_fn <- function() {
    print(x)                   # free variable: looked up lexically
  }
  inner_fn()
}
```

```
outer_fn()      # prints 20 (finds x in outer_fn's environment)
print(x)        # prints 10 (global x untouched)
```

Because inner_fn is DEFINED inside outer_fn, its enclosing environment is fixed at definition time to be outer_fn's local environment — regardless of where inner_fn might later be invoked from. If x were not found there either, R would keep searching further outward: the environment enclosing outer_fn (here, the global environment), and finally the search path of loaded packages. environment(inner_fn) can be inspected directly to confirm which environment a function is bound to.

The superassignment operator <<- exploits this same chain: instead of creating a new local variable, it searches enclosing environments for an existing binding and modifies it there (or creates it in the global environment as a last resort):

```
make_counter <- function() {
  count <- 0
  function() {
    count <<- count + 1     # modifies count in make_counter's environment
    count
  }
}
counter <- make_counter()
counter(); counter(); counter()   # 1, 2, 3 -- state persists
```

This is exactly the mechanism behind closures: the returned inner function retains a permanent reference to make_counter()'s environment (where count lives) even after make_counter() itself has finished executing, so successive calls to counter() correctly see and update the same persistent count. Understanding this environment chain — local, enclosing (lexical), global, and package search path — is essential for correctly reasoning about variable resolution, avoiding accidental global-variable creation, and building stateful closures in R.

Q3.36 *(10 Marks | Bloom's: Apply (L3) | CO3)*

Describe functions as first-class objects in R: passing functions as arguments, returning functions, anonymous functions, and replacement functions. Provide code examples.

In R, a function is a first-class value: it can be assigned to a variable, stored in a list, passed as an argument, and returned from another function, exactly like any other object.

1. Passing functions as arguments (higher-order functions):

```
is_even <- function(n) n %% 2 == 0
sapply(1:6, is_even)          # is_even passed directly into sapply
Filter(is_even, 1:10)         # is_even used as a predicate function
```

2. Returning functions (function factories):

```
make_multiplier <- function(factor) {
  function(x) x * factor        # returns a customised function
}
double <- make_multiplier(2)
triple <- make_multiplier(3)
double(5)   # 10
triple(5)   # 15
```

make_multiplier() returns a new function each time it is called, and that returned function "remembers" its own factor via lexical scoping (a closure) — this is a direct application of function-as-object behaviour.

3. Anonymous functions:

```
sapply(1:5, function(x) x^2)     # classic anonymous function syntax
sapply(1:5, \(x) x^2)            # R 4.1+ shorthand lambda syntax
```

An anonymous function is written inline without being bound to a name — used when a function is needed only once, typically as an argument to apply()/sapply()/Map() and similar higher-order functions.

4. Replacement functions:

```
`second<-` <- function(x, value) { x[2] <- value; x }
v <- c(10,20,30)
second(v) <- 99
v    # 10 99 30
```

A replacement function's name ends in <- ; its last parameter must be named value, and it must return the entire modified object. R rewrites second(v) <- 99 internally as v <- `second<-`(v, 99), giving user-defined objects the same assignment-style syntax as base-R structures such as names(x) <- v. Collectively, treating functions as objects — passing them, returning them, writing them anonymously, and defining custom replacement forms — is the foundation of R's functional-programming capabilities and enables the compact, loop-free pipelines built with apply(), Map(), Reduce() and Filter().

Q3.37 *(10 Marks | Bloom's: Apply (L3) | CO3)*

Design a complete Student Result Processing System in R using data frames: input marks, compute grades using ifelse(), merge with student details, display frequency table and sorted output.

This applied example exercises every major topic of the unit in a single realistic workflow: data frames, ifelse() as a vectorised control statement, merge(), factors/table(), and sorting.

```
# 1. Build the data frame of marks
marks_df <- data.frame(
  id = 1:6,
  marks = c(78, 42, 91, 35, 66, 84)
)

# 2. Compute grades with ifelse() -- vectorised, no explicit loop needed
marks_df$grade <- ifelse(marks_df$marks >= 85, "Distinction",
                  ifelse(marks_df$marks >= 50, "Pass", "Fail"))

# 3. Merge with student/department details
details <- data.frame(
  id = 1:6,
  name = c("Asha","Ravi","Meera","Kabir","Zoya","Dev"),
  dept = c("CSE","AIML","CSE","ECE","AIML","CSE")
)
result <- merge(marks_df, details, by="id")

# 4. Frequency table of grades, and per-department distribution
table(result$grade)
table(result$dept, result$grade)

# 5. Sort and display final report (highest marks first)
result_sorted <- result[order(-result$marks), ]
```

```
print(result_sorted)
```

Step 1 builds the core marks data frame with data.frame(), the natural R structure for tabular records. Step 2 uses nested ifelse() calls — the vectorised equivalent of if-else — to assign a grade band to every student's marks in one pass, without an explicit for loop. Step 3 uses merge() to enrich the marks data with student name and department details from a second table, joined on the common id key (an inner join by default, since every id matches here). Step 4 converts the resulting grade column into a summarised frequency table with table(), and further breaks it down by department using a two-way table — showing, for instance, how many CSE students earned a Distinction. Step 5 uses order() (descending, via the minus sign on result$marks) to rank the final merged data frame by marks for display, producing a clean, sorted report.

This single pipeline — build, transform with ifelse(), merge, summarise with table(), and sort with order() — is exactly the pattern the unit repeatedly returns to (also seen in Q3.43), and models a realistic student-result-processing system end to end.

Q3.38 *(10 Marks | Bloom's: Apply (L3) | CO3)*

Explain recursion in R with examples: factorial, Fibonacci, Tower of Hanoi. Discuss stack depth limitations.

A recursive function solves a problem by calling itself on a smaller version of the same problem, always requiring a base case that stops the recursion and a recursive case that moves toward it.

1. Factorial (linear recursion):

```
factorial <- function(n) {
  if (n <= 1) return(1)         # base case
  n * factorial(n - 1)          # recursive case
}
factorial(5)    # 120
```

2. Fibonacci (branching/tree recursion):

```
fib <- function(n) if (n <= 1) n else fib(n-1) + fib(n-2)
sapply(0:6, fib)    # 0 1 1 2 3 5 8
```

Each fib() call spawns two further calls, so naive recursive Fibonacci grows exponentially with n, re-computing the same sub-values many times over — a classic case where memoization or an iterative rewrite is preferred for larger n.

3. Tower of Hanoi (multi-call, classic algorithmic recursion):

```
hanoi <- function(n, from="A", to="C", via="B") {
  if (n == 1) {
    cat("Move disk 1 from", from, "to", to, "\n")
    return(invisible())
  }
  hanoi(n-1, from, via, to)      # move n-1 disks out of the way
  cat("Move disk", n, "from", from, "to", to, "\n")
  hanoi(n-1, via, to, from)      # move them onto the target
}
hanoi(3)
```

Tower of Hanoi requires exactly two recursive calls per level (move the top n-1 disks aside, move the largest disk, move the n-1 disks back on top), and its recursion depth equals the number of disks n; the total number of moves grows as 2^n - 1, illustrating how naturally some problems decompose recursively even though the growth rate becomes very large.

Stack depth limitations: every pending recursive call remains on R's call stack until it returns, consuming stack memory. R enforces a default limit on nested expression evaluations (visible via options("expressions"), default 5000), and very deep recursion — as with a large n in factorial/Fibonacci, or excessively many Hanoi disks — can trigger a "C stack usage too close to the limit" error and abort the session. Because of this, deeply recursive R functions are often converted to iterative loop-based equivalents, or use tail-recursion-style accumulator patterns and memoization (caching previously computed results) to control both stack depth and repeated computation.

Q3.39 *(10 Marks | Bloom's: Apply (L3) | CO3)*

Explain Math and Simulation in R: distributions (rnorm, runif, rpois), set.seed(), sample(). Design a Monte Carlo simulation to estimate pi and compute error at each step.

R provides built-in random-number generators for the major probability distributions: runif(n, min, max) for the continuous uniform distribution, rnorm(n, mean, sd) for the normal distribution, and rpois(n, lambda) for the discrete Poisson distribution (rbinom() similarly covers the Binomial). sample(x, size, replace) draws a random subset from a vector, and set.seed(x) fixes the random-number generator's internal state so that all subsequent random draws are exactly reproducible.

```
set.seed(42)
sample(1:5, 3)                 # reproducible random draw
runif(3, 0, 10)                 # 3 uniform values in [0,10]
rnorm(3, mean=50, sd=5)          # 3 normal values
rpois(3, lambda=4)               # 3 Poisson counts
```

Monte Carlo estimation of pi, tracking error at each step:

```
set.seed(1)
n_max <- 100000
step <- 5000
x <- runif(n_max); y <- runif(n_max)
inside_cum <- cumsum(x^2 + y^2 <= 1)   # running count inside the circle

checkpoints <- seq(step, n_max, by=step)
for (n in checkpoints) {
  pi_est <- 4 * inside_cum[n] / n
  error  <- abs(pi_est - pi)
  cat(sprintf("n=%6d  pi_est=%.5f  error=%.5f\n", n, pi_est, error))
}
```

The simulation scatters n uniformly random points (x, y) inside the unit square; a point falls inside the inscribed quarter-circle when x^2 + y^2 <= 1. Since the quarter-circle's area relative to the square is pi/4, the running proportion inside_cum[n]/n, multiplied by 4, estimates pi at each checkpoint. cumsum() lets the running inside-count be tracked incrementally without re-simulating, so the error abs(pi_est - pi) can be reported at each step, generally shrinking as n grows — visibly demonstrating the law of large numbers underlying every Monte Carlo method: accuracy improves, and the estimate converges, as the number of random trials increases.

Q3.40 *(10 Marks | Bloom's: Apply (L3) | CO3)*

Describe tools for composing function code in R: do.call(), Reduce(), Map(), Filter(). Build a functional data processing pipeline using these tools.

do.call(), Reduce(), Map() and Filter() let R express data-processing logic without writing explicit for loops, in a compact functional style.

```
do.call(fun, args_list)   # calls fun using arguments assembled dynamically
Reduce(fun, x)            # collapses x to a single accumulated value
Map(fun, x, y)            # applies fun element-wise across parallel vectors/lists
Filter(predicate, x)      # keeps elements of x where predicate(x) is TRUE
```

Pipeline example: given several small data frames of daily sales, combine them, keep only high-value transactions, compute a derived column for each, and total the result.

```
day1 <- data.frame(item=c("A","B"), amount=c(120, 40))
day2 <- data.frame(item=c("C","D"), amount=c(300, 15))
day3 <- data.frame(item=c("E","F"), amount=c(75, 500))
all_days <- list(day1, day2, day3)

# Step 1: combine all daily frames dynamically with do.call + rbind
combined <- do.call(rbind, all_days)

# Step 2: Filter -- keep only transactions above 50
high_value <- combined[Filter(function(i) combined$amount[i] > 50,
                              seq_len(nrow(combined))), ]

# Step 3: Map -- compute a 10% tax amount for each transaction, in parallel
taxes <- Map(function(amt) amt * 0.10, high_value$amount)

# Step 4: Reduce -- collapse all tax amounts into one grand total
total_tax <- Reduce(`+`, taxes)
cat("Total tax collected:", total_tax, "\n")
```

do.call(rbind, all_days) dynamically stacks an arbitrary-length list of data frames into one, without hardcoding how many days exist — exactly the situation do.call() is designed for. Filter() then keeps only the row indices whose amount exceeds the threshold, replacing an explicit for-loop-with-if. Map() applies the tax calculation element-wise across every remaining transaction amount in parallel, returning a list of results. Finally, Reduce(`+`, taxes) collapses that list into a single grand total by successively adding pairs of values. Chaining do.call(), Filter(), Map() and Reduce() this way builds a complete, loop-free, functional data-processing pipeline — combine, filter, transform, aggregate — that is both more concise and, once understood, easier to reason about than the equivalent nested-loop version.

Q3.41 *(10 Marks | Bloom's: Apply (L3) | CO3)*

Explain default values and returning Boolean values from R functions. How do you design robust functions with input validation? Provide a validated statistical function.

Default argument values are specified in a function's signature with name=value, and are substituted automatically whenever the caller omits that argument; they may reference other parameters and are evaluated lazily, only when actually used inside the function body.

```
power <- function(x, n=2) x^n     # n defaults to 2 (square)
power(5)          # 25
power(5, 3)       # 125
```

Functions that return Boolean values directly (e.g. is_valid <- function(x) x > 0 & !is.na(x)) can be used straight inside if() conditions or as predicate functions for Filter() and similar higher-order functions, because their result is already a logical TRUE/FALSE ready for direct use.

Designing robust functions means validating arguments BEFORE performing the main computation, and either returning an informative FALSE/NA or halting with stop() and a clear message, rather than letting the function fail silently or produce a misleading numeric result:

```
safe_mean <- function(x, na.rm = FALSE) {
  # --- input validation ---
  if (!is.numeric(x)) {
    stop("safe_mean(): 'x' must be a numeric vector")
  }
  if (length(x) == 0) {
    warning("safe_mean(): 'x' is empty, returning NA")
    return(NA)
  }
  if (!is.logical(na.rm)) {
    stop("safe_mean(): 'na.rm' must be TRUE or FALSE")
  }
  # --- main computation ---
  mean(x, na.rm = na.rm)
}

safe_mean(c(4, 8, 15, 16, 23, NA), na.rm = TRUE)  # valid call -> works
safe_mean("not numeric")                          # invalid -> stop() with message
```

safe_mean() first checks that x is numeric and that na.rm is a proper logical value using is.numeric()/is.logical(), each guarded by a default argument (na.rm=FALSE) so the function remains easy to call for the common case. It checks length(x)==0 as an edge case, warning and returning NA gracefully rather than letting mean() silently return NaN. Only after all checks pass does it perform the actual computation. This layered pattern — sensible defaults, explicit type/edge-case checks, clear stop()/warning() messages, then the core logic — is the standard approach for writing production-quality, self-documenting R functions that fail loudly and informatively rather than silently producing wrong results.

Q3.42 *(10 Marks | Bloom's: Understand (L2) | CO3)*

Compare while, for, and repeat loops in R with respect to: syntax, use cases, break/next usage. Give a practical example for each.

R's three loop constructs share the same body syntax but differ fundamentally in how they decide when to stop.

```
# for: fixed sequence, known iteration count
for (i in 1:5) print(i^2)

# while: condition checked BEFORE each iteration; may run zero times
n <- 5
while (n > 0) { print(n); n <- n - 1 }

# repeat: no condition in header; guarantees at least one run; needs break
x <- 1
repeat {
  print(x)
  x <- x + 1
  if (x > 5) break
}
```

Syntax: for (var in sequence) { ... } iterates the loop variable through a known vector, list, or range, one value per pass. while (condition) { ... } re-tests a logical condition before every pass and stops the moment it becomes FALSE. repeat { ... } has no condition at all in its header — it always runs the body first and must be stopped manually from inside, typically via an if()-guarded break.

Use cases: for is best when the exact number of iterations (or the exact sequence to walk through) is known in advance — e.g. processing every row of a data frame or every element of a vector. while suits condition-controlled repetition where the number of iterations is not known beforehand but a clear stopping condition exists, e.g. "keep halving a value until it drops below 1". repeat is used for indefinite repetition, especially retry logic or event-driven loops ("keep trying until success, or until an external event occurs") that must run at least once regardless of any starting condition.

break/next usage: break immediately terminates the innermost loop and hands control to the code after it — essential in repeat, since without it the loop would never stop, but equally usable in for and while to exit early once a goal is met (e.g. stop searching once a match is found). next skips only the remainder of the CURRENT iteration and jumps to the next pass, useful for filtering out unwanted values mid-loop (e.g. skipping NA records) in any of the three loop types without terminating the loop altogether. Careful placement of both, especially inside while and repeat, is essential to avoid accidentally writing an infinite loop.

Q3.43 *(10 Marks | Bloom's: Apply (L3) | CO3)*

Write an R program to: (i) create a data frame of 50 employee records, (ii) filter by department, (iii) compute salary statistics per department, (iv) merge with department data, (v) generate frequency table.

```
# (i) Create a data frame of 50 employee records
set.seed(10)
employees <- data.frame(
  emp_id = 1:50,
  name   = paste0("Emp", 1:50),
  dept_id = sample(1:4, 50, replace = TRUE),
  salary  = round(rnorm(50, mean = 45000, sd = 8000), 0)
)
head(employees)

# (ii) Filter by department (example: department 2 only)
dept2_employees <- employees[employees$dept_id == 2, ]
print(dept2_employees)

# (iii) Compute salary statistics per department
salary_stats <- tapply(employees$salary, employees$dept_id,
                       function(x) c(mean = mean(x), min = min(x),
                                     max = max(x), sd = sd(x)))
print(salary_stats)

# (iv) Merge with department data
dept_names <- data.frame(
  dept_id = 1:4,
  dept_name = c("CSE", "AIML", "ECE", "MECH")
)
employees_full <- merge(employees, dept_names, by = "dept_id")

# (v) Generate frequency table (headcount per department)
table(employees_full$dept_name)
```

(i) data.frame() builds 50 synthetic employee records with an id, name, a randomly-assigned dept_id (1-4), and a salary drawn from a normal distribution using rnorm(); set.seed() makes the generated data reproducible for grading.

(ii) A logical condition on dept_id inside [,] filters the frame down to only the rows belonging to a chosen department — the standard matrix-style row-filtering pattern for data frames.

(iii) tapply(x, INDEX, FUN) splits the salary vector by the dept_id grouping factor and applies a custom summary function to each group, returning per-department mean, minimum, maximum and standard deviation in one call — the factor-aware apply function is the natural tool for "statistics per group".

(iv) merge() joins the numeric dept_id key against a small lookup table of department names, by default performing an inner join, turning the raw dept_id codes into a readable dept_name column throughout the employee data.

(v) Finally, table() on the merged dept_name column produces a simple frequency count — the headcount in each named department — completing a realistic end-to-end HR-analytics-style workflow that exercises data frame creation, filtering, grouped statistics, merging, and frequency tabulation together, exactly the applied combination emphasised throughout Unit III.

Q3.44 *(10 Marks | Bloom's: Understand (L2) | CO3)*

Describe replacement functions in R (names<-, levels<-, dim<-). Explain how they work internally. Design a custom replacement function.

A replacement function is any function whose name ends in <- and which is used to MODIFY part of an object through ordinary-looking assignment syntax, e.g. names(x) <- v, levels(f) <- new_labels, or dim(m) <- c(2,3).

How they work internally: when R parses an expression such as names(x) <- v, it rewrites it behind the scenes as x <- `names<-`(x, value = v) — that is, it calls the function literally named names<- (backtick-quoted because <- is not normally legal inside an identifier), passing the original object x and the right-hand-side value as the value argument, and then reassigns the ENTIRE returned result back to x. Two rules are enforced by this mechanism: the replacement function's last formal parameter must be named value, and the function must return the complete, modified object rather than just the changed portion, because that returned object wholly replaces x.

```
x <- c(a=1, b=2, c=3)
names(x) <- c("p","q","r")     # internally: x <- `names<-`(x, c("p","q","r"))

f <- factor(c("lo","hi","lo"))
levels(f) <- c("High","Low")    # renames the existing levels, in level order

m <- 1:6
dim(m) <- c(2,3)                # reshapes m into a 2x3 matrix
```

names<-, levels<- and dim<- are built-in replacement functions that follow exactly this pattern: names<- rewrites an object's names attribute, levels<- renames (or reorders) a factor's category labels, and dim<- reshapes a vector by setting its dim attribute, turning it into a matrix/array.

Custom replacement function:

```
`second<-` <- function(x, value) {
  x[2] <- value      # modify the second element
  x                  # must return the whole object
}

v <- c(10, 20, 30)
second(v) <- 99      # equivalent to: v <- `second<-`(v, 99)
print(v)             # 10 99 30
```

Here second<- is a user-defined replacement function that lets any vector adopt the natural syntax second(v) <- 99 to overwrite its second element, exactly mirroring how the built-in replacement functions behave — R itself makes no distinction between built-in and user-defined replacement functions, both are resolved through the same <- -rewriting mechanism, which is why understanding this internal rewrite is key to designing correct custom ones.

Q3.45 *(10 Marks | Bloom's: Apply (L3) | CO3)*

Explain Boolean operators in R (&, |, !, &&, ||, xor). Explain short-circuit evaluation. Write a data validation function using multiple operators.

R distinguishes between vectorised and scalar (short-circuit) Boolean operators.

```
c(TRUE,FALSE) & c(TRUE,TRUE)     # vectorised AND -> TRUE FALSE
c(TRUE,FALSE) | c(FALSE,FALSE)   # vectorised OR  -> TRUE FALSE
!c(TRUE,FALSE)                    # NOT           -> FALSE TRUE
xor(TRUE, FALSE)                  # exactly one TRUE -> TRUE

TRUE && FALSE     # scalar AND, short-circuits
FALSE || TRUE     # scalar OR,  short-circuits
```

& and | operate element-wise across entire vectors, returning a logical vector of the same length — the correct choice for filtering data frame rows, since every row's condition must be evaluated independently. ! simply negates a logical vector element-wise. xor(a, b) returns TRUE only when exactly one of a and b is TRUE (an exclusive-or), useful for detecting mismatches between two flags.

&& and || instead compare only single (length-1) logical values, and are required inside if() conditions (using a vector there triggers a warning/error in modern R). Short-circuit evaluation means these operators evaluate their operands left to right and stop as soon as the overall result is already certain: with &&, if the first operand is FALSE the whole expression must be FALSE, so the second operand is never evaluated at all; with ||, if the first operand is TRUE, the second is skipped. This makes && particularly valuable for GUARD conditions, safely checking a precondition before an operation that could otherwise error:

```
x <- numeric(0)
if (length(x) > 0 && x[1] > 5) {
  print("safe")
}   # x[1] is never evaluated because length(x) > 0 is already FALSE -- no error
```

Data validation function using multiple Boolean operators:

```
validate_record <- function(age, income, has_id) {
  age_ok    <- !is.na(age) & age >= 18 & age <= 65
  income_ok <- !is.na(income) & income > 0
  valid <- age_ok & income_ok & has_id
  # xor flags a specific inconsistency: has an ID but fails the other checks
  suspicious <- xor(has_id, (age_ok & income_ok))
  list(valid = valid, suspicious = suspicious)
}
validate_record(25, 30000, TRUE)   # valid=TRUE,  suspicious=FALSE
validate_record(15, 30000, TRUE)   # valid=FALSE, suspicious=TRUE
```

This function combines & for element-wise range/positivity checks (safe across an entire vector of records), ! for negating the is.na() test, and xor() to flag a specific kind of inconsistency (holding an ID despite otherwise failing

validation) — demonstrating how the vectorised operators, the negation operator, and xor() work together in a realistic validation routine, distinct from the scalar && / || used purely for control-flow guards.

Q3.46 *(10 Marks | Bloom's: Apply (L3) | CO3)*

Explain the use of factors in data analysis. How do factor levels affect grouping operations in table() and tapply()? Provide examples.

Factors are R's structure for representing categorical (grouping) variables — department, gender, grade band, risk category, and so on — storing the data as integer codes plus a levels attribute of category labels. In data analysis, factors are what let R recognise a column as "groups to summarise by" rather than free text or arbitrary numbers, and they drive grouping operations, plotting axes, and statistical modelling (e.g. as predictors in regression).

```
dept <- factor(c("CSE","AIML","CSE","ECE","AIML","CSE"))
salary <- c(45000, 52000, 48000, 41000, 55000, 47000)

levels(dept)             # "AIML" "CSE" "ECE" (alphabetical by default)
table(dept)               # frequency count per level
tapply(salary, dept, mean)  # mean salary, grouped by department level
```

Effect of factor levels on table(): table() counts observations against EVERY level currently defined on the factor, even a level that happens to have zero occurrences in the actual data will still appear in the output with a count of 0 (unless removed first with droplevels()). This means the set and order of levels(dept) directly determines which categories appear, and in what order, in the table() output — a factor built from a subset of data can still "remember" levels no longer present, producing misleading zero-count rows unless droplevels() is applied.

Effect of factor levels on tapply(): tapply(x, INDEX, FUN) splits the numeric vector x according to the levels of the grouping factor INDEX and applies FUN separately to each group, again returning a result indexed by every level of the factor (including a NA entry for any level with no data). The output order is always determined by levels(INDEX), not by the order values first appear in the data — so if dept had been created with an explicit levels= or as an ordered factor with a custom rank, tapply()'s output rows would follow that specified order instead of alphabetical order.

In short, a factor's level set and level order act as the fixed "grouping key" for both table() and tapply(): they determine which groups are shown, their display order, and whether unused categories silently appear with zero/NA results — making explicit management of levels() (and periodic use of droplevels()) an important part of correct grouped analysis in R.

Q3.47 *(10 Marks | Bloom's: Apply (L3) | CO3)*

Design an R program to simulate a coin-toss experiment 10,000 times using sample(). Compute empirical probability and compare with theoretical. Visualize with a histogram.

```
set.seed(123)
n_tosses <- 10000

# Simulate: sample() draws Heads/Tails with equal probability, with replacement
tosses <- sample(c("Heads","Tails"), size = n_tosses, replace = TRUE)

# Empirical probability of Heads
freq <- table(tosses)
print(freq)
empirical_p_heads <- freq["Heads"] / n_tosses
theoretical_p <- 0.5
```

```
cat("Empirical P(Heads):", empirical_p_heads, "\n")
cat("Theoretical P(Heads):", theoretical_p, "\n")
cat("Absolute error:", abs(empirical_p_heads - theoretical_p), "\n")

# Track running proportion of Heads to see convergence
is_heads <- as.numeric(tosses == "Heads")
running_prop <- cumsum(is_heads) / seq_len(n_tosses)

plot(running_prop, type = "l", col = "blue",
     xlab = "Number of tosses", ylab = "Proportion of Heads",
     main = "Convergence of Empirical Probability to 0.5")
abline(h = 0.5, col = "red", lty = 2)

# Histogram-style bar chart of outcome counts
barplot(freq, col = c("skyblue","salmon"),
        main = "Coin Toss Outcomes (n = 10,000)", ylab = "Frequency")
```

sample(c("Heads","Tails"), size=n_tosses, replace=TRUE) simulates 10,000 independent fair coin tosses, each equally likely to be Heads or Tails, with replace=TRUE ensuring every toss is drawn independently of the last (as a real coin toss would be). table(tosses) then tallies exactly how many of each outcome occurred, from which the empirical probability of Heads (freq["Heads"]/n_tosses) is computed directly as observed relative frequency.

Because a fair coin has a theoretical probability of exactly 0.5 for Heads, comparing the two shows how close the simulation lands to theory — with 10,000 trials, the empirical probability typically falls within about 1% of 0.5, and the gap (absolute error) is reported explicitly. Tracking cumsum(is_heads)/seq_len(n_tosses) produces a running proportion of Heads after each toss, and plotting it (with abline(h=0.5) as the theoretical reference line) visually demonstrates the law of large numbers: the early tosses fluctuate noticeably, but the running proportion settles and converges toward 0.5 as the number of tosses grows. Since bar counts of a two-category factor are the natural visualisation here, a barplot() of the frequency table serves as the histogram-style chart, showing the near-equal split between Heads and Tails outcomes across the full 10,000-toss simulation.

Q3.48 *(10 Marks | Bloom's: Apply (L3) | CO3)*

Explain the functional programming approach in R using Map(), Reduce(), and Filter(). Demonstrate a complete data pipeline replacing explicit loops.

R supports a functional-programming style in which data transformations are expressed by composing small, reusable functions instead of writing explicit for loops with manual accumulator variables. Map(), Reduce() and Filter() are the three core building blocks of this style.

```
Map(fun, x, y, ...)     # apply fun element-wise, in parallel, over one or more inputs ->
returns a list
Reduce(fun, x, init)    # successively combine elements of x with a binary function ->
single value
Filter(predicate, x)    # keep only elements of x for which predicate(x) is TRUE
```

Complete pipeline: given raw daily transaction amounts, remove invalid entries, apply a currency conversion, and compute the grand total — all without an explicit loop.

```
raw_amounts <- list(120, -5, 340, NA, 80, 0, 275)

# Step 1: Filter -- discard invalid entries (NA, zero, or negative)
valid_amounts <- Filter(function(a) !is.na(a) && a > 0, raw_amounts)
print(valid_amounts)

# Step 2: Map -- convert each valid amount from USD to INR (rate = 83)
```

```
inr_amounts <- Map(function(a) a * 83, valid_amounts)
print(inr_amounts)

# Step 3: Reduce -- sum all converted amounts into one grand total
grand_total <- Reduce(function(acc, x) acc + x, inr_amounts, 0)
cat("Grand total (INR):", grand_total, "\n")

# Equivalent one-line composition
total_alt <- Reduce(`+`, Map(function(a) a*83,
                     Filter(function(a) !is.na(a) && a > 0, raw_amounts)))
identical(grand_total, total_alt)   # TRUE
```

Filter() replaces what would otherwise be a for-loop with an if-condition and a manually grown result vector — it declares the intent ("keep only positive, non-missing amounts") directly. Map() replaces a loop that would apply the same currency-conversion formula to every element and collect the results, instead applying the transformation element-wise in one call and returning a list. Reduce() replaces the running-total accumulator pattern (total <- 0; for(...) total <- total + x) by expressing "combine everything with +" as a single declarative call, with an explicit initial value of 0 to make the starting point clear.

Chaining Filter() -> Map() -> Reduce(), as shown in the one-line composition, builds a complete filter-transform-aggregate pipeline that reads top-to-bottom in the same order the data actually flows, is easier to test piece by piece, and avoids the mutable loop-counter and accumulator-variable bugs that are common in the equivalent hand-written for-loop version.

Q3.49 *(10 Marks | Bloom's: Apply (L3) | CO3)*

Write an R program demonstrating: (i) writing upstairs with <<-, (ii) closure-based counter, (iii) memoization using environment. Explain closures in R.

A closure in R is a function bundled together with the environment in which it was created; the inner function retains permanent access to that enclosing environment's variables even after the outer function that created it has already finished running. This is possible because of R's lexical scoping: a function always looks up free variables in the environment where it was DEFINED, not where it is called from.

(i) Writing upstairs with <<-:

```
demo_upstairs <- function() {
  total <- 0
  update <- function(amount) {
    total <<- total + amount   # writes to 'total' in the enclosing environment
    total
  }
  update
}
acc <- demo_upstairs()
acc(100); acc(50); acc(25)     # 100, 150, 175 -- total persists across calls
```

<<- searches enclosing environments for an existing total and assigns there rather than creating a fresh local copy, so successive calls to acc() accumulate rather than reset.

(ii) Closure-based counter:

```
make_counter <- function(start = 0) {
  count <- start
  function() {
    count <<- count + 1
```

```
    count
  }
}
counter1 <- make_counter()
counter2 <- make_counter(100)
counter1(); counter1()   # 1, 2
counter2()               # 101 -- independent state from counter1
```

Each call to make_counter() creates a NEW, separate environment holding its own count, so counter1 and counter2 are independent closures with independent state — demonstrating that closures capture their own private environment rather than sharing one global variable.

(iii) Memoization using an environment:

```
make_memo_fib <- function() {
  cache <- new.env()
  function(n) {
    key <- as.character(n)
    if (!is.null(cache[[key]])) return(cache[[key]])   # cache hit
    result <- if (n <= 1) n else Recall(n-1) + Recall(n-2)
    cache[[key]] <- result                              # store for reuse
    result
  }
}
memo_fib <- make_memo_fib()
sapply(0:10, memo_fib)   # computed once per n, reused thereafter
```

Here a dedicated environment cache (created with new.env(), which behaves like a mutable hash map) stores previously computed Fibonacci results keyed by n; the returned closure checks the cache before recursing, avoiding the exponential re-computation of naive recursive Fibonacci and dramatically speeding up repeated calls.

In all three examples the same underlying mechanism is at work: a returned inner function keeps a live reference to the local environment of the outer function that created it, and <<- (or direct environment manipulation via [[ ]] on an environment object) is the tool used to modify that retained state across separate calls — this is precisely what makes closures useful for counters, accumulators, and memoized caches in R.

Q3.50 *(10 Marks | Bloom's: Apply (L3) | CO3)*

Create a simulation in R to model a random walk of 1000 steps: track position, compute return probability, and plot the trajectory.

```
set.seed(7)
n_steps <- 1000

# Each step is +1 or -1 with equal probability
steps <- sample(c(-1, 1), size = n_steps, replace = TRUE)

# Track position: cumulative sum of steps gives position after each step
position <- cumsum(steps)

cat("Final position after", n_steps, "steps:", tail(position, 1), "\n")
cat("Maximum distance reached:", max(abs(position)), "\n")

# Compute (empirical) return probability: how often the walk revisits 0
returns_to_zero <- sum(position == 0)
return_probability <- returns_to_zero / n_steps
```

```
cat("Number of returns to origin:", returns_to_zero, "\n")
cat("Empirical return probability:", return_probability, "\n")

# Plot the trajectory
plot(0:n_steps, c(0, position), type = "l", col = "blue",
     xlab = "Step number", ylab = "Position",
     main = "1-D Random Walk (1000 steps)")
abline(h = 0, col = "red", lty = 2)
```

sample(c(-1,1), n_steps, replace=TRUE) simulates each of the 1000 individual steps of the walk, moving either +1 or -1 with equal probability, independently of all previous steps — exactly the defining property of a simple symmetric random walk. cumsum(steps) then converts the sequence of individual +1/-1 moves into the actual POSITION after each step, since position at step k is simply the running total of the first k moves.

The final position (tail(position,1)) shows where the walk ends up after all 1000 steps, and max(abs(position)) reports how far it wandered from the origin at its most extreme point — both are typical descriptive statistics for a random walk. The return probability is estimated empirically by counting how many of the 1000 recorded positions are exactly 0 (position==0) and dividing by the total number of steps; this gives the observed fraction of time the walk revisits its starting point, which can then be compared against the well-known theoretical result that a 1-D symmetric random walk is recurrent (it returns to the origin infinitely often as steps -> infinity), though the return probability at any finite step count remains modest.

Finally, plot(0:n_steps, c(0,position), type="l") draws the full trajectory of the walk as a connected line from the origin through every subsequent position, with abline(h=0) marking the starting level for visual reference — this line plot is the standard way to visualise how a random walk drifts, oscillates, and occasionally crosses back over zero across its full length, and follows exactly the same simulate -> count -> estimate pattern used earlier for the pi and coin-toss Monte Carlo simulations.